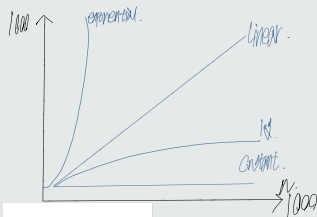


Workspace 'Week 1' in '6.006'

Page 1 (row 1, column 1)

input	constant	logarithmic	linear	log-linear	quadratic	polynomial	exponential
n	$\Theta(1)$	$\Theta(\log n)$	$\Theta(n)$	$\Theta(n \log n)$	$\Theta(n^2)$	$\Theta(n^k)$	$2^{\Theta(n)}$
1000	1	≈ 10	1000	$\approx 10,000$	1,000,000	1,000 ^k	$2^{1000} \approx 10^{301}$
Time	1 ns	10 ns	1 μ s	10 μ s	1 ms	10 ^k s	10 ²⁸ millennia

这些时间复杂度
相差多少倍



Lecture 2: Data Structures

Interface is data structures

- What you want to do? How you do it?
- Representation
- What data on ops - How to store data.
- Algorithms to support operations
- What the operation are important & what they mean
- Problem - Solution

2. main interface

- set

- sequence

[5, 2, 1, 7]

2. main data structures approaches

- Arrays

- pointer based

Sequence Interface

Key: Word from model of computation.

Collection (natural): static array $A[i] = a_i$

- $O(1)$ per $\text{get_at} / \text{set_at}$

- $O(n)$ per $\text{build} / \text{resize}$

- memory: array of w -bit words

- "array": consecutive chunk of memory.

$\Rightarrow \text{array}[i] = \text{memory}[\text{address}(\text{array}) + i]$

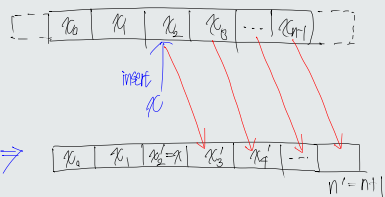
$\Rightarrow \text{array access is } O(1) \text{ time}$

Assume $w \geq n$.

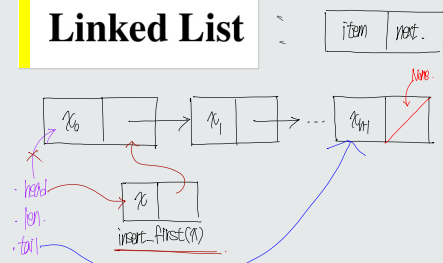
$\Rightarrow$ Memory allocation model: allocate array of size n in $O(n)$ time

space = $O(\text{time})$

Dynamic sequence interface:



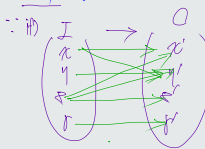
Linked List



Problem

$f: I \rightarrow O$

binary relation



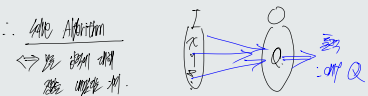
$f(x) = x^2$
 $f(x) = 4^x$

这些时间复杂度相差多少倍

Algorithm

$f: I \rightarrow O$

1st. problem size
2nd. input size
3rd. output size



4th. time complexity

1st. time complexity
2nd. space complexity

3rd. time complexity

4th. time complexity
5th. space complexity

6th. time complexity

7th. time complexity
8th. space complexity

Correctness

$\Rightarrow Q$ is the correct solution
for the problem
Is the solution correct for the problem?

(base) case: base case

(inductive) case: inductive case

is the solution correct for the problem?

is the solution correct for the problem?

is the solution correct for the problem?

is the solution correct for the problem?

is the solution correct for the problem?

is the solution correct for the problem?

is the solution correct for the problem?

is the solution correct for the problem?

is the solution correct for the problem?

Page 3 (row 3, column 1)

∴ 证明 1 略 2 略 3 略 4 略

得吧：瑞利

$$\Leftrightarrow \text{ket} : \underline{\text{ket}}$$

外 經 訓 引 訓 何 訓

	Container	
	<code>build(k)</code>	given an iterable <code>x</code> , build sequence from items in <code>x</code>
	<code>len()</code>	return the number of stored items
Static	<code>find(k)</code>	return the stored item with key <code>k</code>
Dynamic	<code>insert(k)</code>	add <code>x</code> to set (replace item with key <code>x</code> , <code>key</code> if one already exists)
	<code>delete(k)</code>	remove and return the stored item with key <code>k</code>
	<code>iter_min()</code>	return the stored items one-by-one in order in key order
	<code>find_min()</code>	return the stored item with smallest key
	<code>find_max()</code>	return the stored item with largest key
	<code>find_next(k)</code>	return the stored item with smallest key larger than <code>k</code>
	<code>find_prev(k)</code>	return the stored item with largest key smaller than <code>k</code>

∴ Army 防衛隊

既新也壯

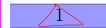
ii) 咽部新生物

· 晚 秋

· 继续创作

$$\therefore \text{该复数的模} = \frac{1000 + i \cdot 1000}{1000 - 1000i} : 1000$$

```
get_at (i)
set_at (i, x)
```



• WEG
: 魏 功 功 魏 功

Q22) 221 的 insert: 根 2 的左孩子 221 的右孩子 ... 221 的 (n)

(1632) 301 201 (1041-1042) 1041 1042 1043 1044 1045 1046 1047 1048 1049 1050 1051 1052 1053 1054 1055 1056 1057 1058 1059 1060 1061 1062 1063 1064 1065 1066 1067 1068 1069 1070 1071 1072 1073 1074 1075 1076 1077 1078 1079 1080 1081 1082 1083 1084 1085 1086 1087 1088 1089 1090 1091 1092 1093 1094 1095 1096 1097 1098 1099 1100 1101 1102 1103 1104 1105 1106 1107 1108 1109 1110 1111 1112 1113 1114 1115 1116 1117 1118 1119 1120 1121 1122 1123 1124 1125 1126 1127 1128 1129 1130 1131 1132 1133 1134 1135 1136 1137 1138 1139 1140 1141 1142 1143 1144 1145 1146 1147 1148 1149 1150 1151 1152 1153 1154 1155 1156 1157 1158 1159 1160 1161 1162 1163 1164 1165 1166 1167 1168 1169 1170 1171 1172 1173 1174 1175 1176 1177 1178 1179 1180 1181 1182 1183 1184 1185 1186 1187 1188 1189 1190 1191 1192 1193 1194 1195 1196 1197 1198 1199 1200 1201 1202 1203 1204 1205 1206 1207 1208 1209 1210 1211 1212 1213 1214 1215 1216 1217 1218 1219 1220 1221 1222 1223 1224 1225 1226 1227 1228 1229 1230 1231 1232 1233 1234 1235 1236 1237 1238 1239 1240 1241 1242 1243 1244 1245 1246 1247 1248 1249 1250 1251 1252 1253 1254 1255 1256 1257 1258 1259 1260 1261 1262 1263 1264 1265 1266 1267 1268 1269 1270 1271 1272 1273 1274 1275 1276 1277 1278 1279 1280 1281 1282 1283 1284 1285 1286 1287 1288 1289 1290 1291 1292 1293 1294 1295 1296 1297 1298 1299 1300 1301 1302 1303 1304 1305 1306 1307 1308 1309 1310 1311 1312 1313 1314 1315 1316 1317 1318 1319 1320 1321 1322 1323 1324 1325 1326 1327 1328 1329 1330 1331 1332 1333 1334 1335 1336 1337 1338 1339 1340 1341 1342 1343 1344 1345 1346 1347 1348 1349 1350 1351 1352 1353 1354 1355 1356 1357 1358 1359 1360 1361 1362 1363 1364 1365 1366 1367 1368 1369 1370 1371 1372 1373 1374 1375 1376 1377 1378 1379 1380 1381 1382 1383 1384 1385 1386 1387 1388 1389 1390 1391 1392 1393 1394 1395 1396 1397 1398 1399 1400 1401 1402 1403 1404 1405 1406 1407 1408 1409 1410 1411 1412 1413 1414 1415 1416 1417 1418 1419 1420 1421 1422 1423 1424 1425 1426 1427 1428 1429 1430 1431 1432 1433 1434 1435 1436 1437 1438 1439 1440 1441 1442 1443 1444 1445 1446 1447 1448 1449 1450 1451 1452 1453 1454 1455 1456 1457 1458 1459 1460 1461 1462 1463 1464 1465 1466 1467 1468 1469 1470 1471 1472 1473 1474 1475 1476 1477 1478 1479 1480 1481 1482 1483 1484 1485 1486 1487 1488 1489 1490 1491 1492 1493 1494 1495 1496 1497 1498 1499 1500 1501 1502 1503 1504 1505 1506 1507 1508 1509 1510 1511 1512 1513 1514 1515 1516 1517 1518 1519 1520 1521 1522 1523 1524 1525 1526 1527 1528 1529 1530 1531 1532 1533 1534 1535 1536 1537 1538 1539 1540 1541 1542 1543 1544 1545 1546 1547 1548 1549 1550 1551 1552 1553 1554 1555 1556 1557 1558 1559 1560 1561 1562 1563 1564 1565 1566 1567 1568 1569 1570 1571 1572 1573 1574 1575 1576 1577 1578 1579 1580 1581 1582 1583 1584 1585 1586 1587 1588 1589 1590 1591 1592 1593 1594 1595 1596 1597 1598 1599 1600 1601 1602 1603 1604 1605 1606 1607 1608 1609 1610 1611 1612 1613 1614 1615 1616 1617 1618 1619 1620 1621 1622 1623 1624 1625 1626 1627 1628 1629 1630 1631 1632 1633 1634 1635 1636 1637 1638 1639 1640 1641 1642 1643 1644 1645 1646 1647 1648 1649 1650 1651 1652 1653 1654 1655 1656 1657 1658 1659 1660 1661 1662 1663 1664 1665 1666 1667 1668 1669 1670 1671 1672 1673 1674 1675 1676 1677 1678 1679 1680 1681 1682 1683 1684 1685 1686 1687 1688 1689 1690 1691 1692 1693 1694 1695 1696 1697 1698 1699 1700 1701 1702 1703 1704 1705 1706 1707 1708 1709 1710 1711 1712 1713 1714 1715 1716 1717 1718 1719 1720 1721 1722 1723 1724 1725 1726 1727 1728 1729 1730 1731 1732 1733 1734 1735 1736 1737 1738 1739 1740 1741 1742 1743 1744 1745 1746 1747 1748 1749 1750 1751 1752 1753 1754 1755 1756 1757 1758 1759 1760 1761 1762 1763 1764 1765 1766 1767 1768 1769 1770 1771 1772 1773 1774 1775 1776 1777 1778 1779 1780 1781 1782 1783 1784 1785 1786 1787 1788 1789 1790 1791 1792 1793 1794 1795 1796 1797 1798 1799 1800 1801 1802 1803 1804 1805 1806 1807 1808 1809 1810 1811 1812 1813 1814 1815 1816 1817 1818 1819 1820 1821 1822 1823 1824 1825 1826 1827 1828 1829 1830 1831 1832 1833 1834 1835 1836 1837 1838 1839 1840 1841 1842 1843 1844 1845 1846 1847 1848 1849 1850 1851 1852 1853 1854

∴ 正则语言 ($\neq$ 正则列表)

∴ 正则语言 ($\neq$ 正则列表)

- 將 model 變為 data 的 model

• 有 KOH 与 H_2SO_4

· 陳在 臨江 州 順德 王姓
陳 瑞 府。

· 16 吨至 18 吨 20 吨

∴ Amy 和 她 的 姊 姊
她 姊 姊 的 姊 姊 的 姊 姊
她 姊 姊 的 姊 姊

== Python 呢

⇒ 此 处 初 始 值 为 0
: 0 1 2 3 4 5 6 7 8 9
11 12 13 14 15
for i in range(0, 16):
 if i % 2 == 1: S[i] = 0

Amortized Analysis (amortized)

: 0 1 2 3 4 5 6 7 8 9
T(n) 的 值 为 0 1 2 3 4 5 6 7 8 9

Lecture 2: Data Structures

Data Structure Interfaces

- A **data structure** is a way to store data, with algorithms that support **operations** on the data
- Collection of supported operations is called an **interface** (also API or ADT)
- Interface is a **specification**: what operations are supported (the **problem!**)
- Data structure is a **representation**: how operations are supported (the **solution!**)
- In this class, two main interfaces: **Sequence** and **Set**

Sequence Interface (L02, L07)

- Maintain a sequence of items (order is **extrinsic**)
- Ex: $(x_0, x_1, x_2, \dots, x_{n-1})$ (zero indexing)
- (use n to denote the number of items stored in the data structure)
- Supports sequence operations:

Container	build(x) len()	given an iterable x , build sequence from items in x return the number of stored items
Static	iter_seq() get_at(i) set_at(i, x)	return the stored items one-by-one in sequence order return the i^{th} item replace the i^{th} item with x
Dynamic	insert_at(i, x) delete_at(i) insert_first(x) delete_first() insert_last(x) delete_last()	add x as the i^{th} item remove and return the i^{th} item add x as the first item remove and return the first item add x as the last item remove and return the last item

- Special case interfaces:

stack	insert_last(x) and delete_last()
queue	insert_first(x) and delete_first()

Set Interface (L03-L08)

- Sequence about **extrinsic** order, set is about **intrinsic** order
- Maintain a set of items having **unique keys** (e.g., item x has key $x.key$)
- (Set or multi-set? We restrict to unique keys for now.)
- Often we let key of an item be the item itself, but may want to store more info than just key
- Supports set operations:

Container	<code>build(X)</code> <code>len()</code>	given an iterable x , build sequence from items in x return the number of stored items
Static	<code>find(k)</code>	return the stored item with key k
Dynamic	<code>insert(x)</code> <code>delete(k)</code>	add x to set (replace item with key $x.key$ if one already exists) remove and return the stored item with key k
Order	<code>iter_ord()</code> <code>find_min()</code> <code>find_max()</code> <code>find_next(k)</code> <code>find_prev(k)</code>	return the stored items one-by-one in key order return the stored item with smallest key return the stored item with largest key return the stored item with smallest key larger than k return the stored item with largest key smaller than k

- Special case interfaces:
 - dictionary** | set without the Order operations
- In recitation, you will be asked to implement a Set, given a Sequence data structure.

Array Sequence

- Array is great for static operations! `get_at(i)` and `set_at(i, x)` in $\Theta(1)$ time!
- But not so great at dynamic operations...
- (For consistency, we maintain the invariant that array is full)
- Then inserting and removing items requires:
 - reallocating the array
 - shifting all items after the modified item

Data Structure	Operation, Worst Case $O(\cdot)$				
	Container	Static	Dynamic		
	<code>build(X)</code>	<code>get_at(i)</code> <code>set_at(i, x)</code>	<code>insert_first(x)</code> <code>delete_first()</code>	<code>insert_last(x)</code> <code>delete_last()</code>	<code>insert_at(i, x)</code> <code>delete_at(i)</code>
Array	n	1	n	n	n

Linked List Sequence

- Pointer data structure (this is **not** related to a Python “list”)
- Each item stored in a **node** which contains a pointer to the next node in sequence
- Each node has two fields: `node.item` and `node.next`
- Can manipulate nodes simply by relinking pointers!
- Maintain pointers to the first node in sequence (called the head)
- Can now insert and delete from the front in $\Theta(1)$ time! Yay!
- (Inserting/deleting efficiently from back is also possible; you will do this in PS1)
- But now `get_at(i)` and `set_at(i, x)` each take $O(n)$ time... :(
- Can we get the best of both worlds? Yes! (Kind of...)

Data Structure	Operation, Worst Case $O(\cdot)$				
	Container	Static	Dynamic		
	<code>build(X)</code>	<code>get_at(i)</code> <code>set_at(i, x)</code>	<code>insert_first(x)</code> <code>delete_first()</code>	<code>insert_last(x)</code> <code>delete_last()</code>	<code>insert_at(i, x)</code> <code>delete_at(i)</code>
Linked List	n	n	1	n	n

Dynamic Array Sequence

- Make an array efficient for **last** dynamic operations
- Python “list” is a dynamic array
- **Idea!** Allocate extra space so reallocation does not occur with every dynamic operation
- **Fill ratio:** $0 \leq r \leq 1$ the ratio of items to space
- Whenever array is full ($r = 1$), allocate $\Theta(n)$ extra space at end to fill ratio r_i (e.g., $1/2$)
- Will have to insert $\Theta(n)$ items before the next reallocation
- A single operation can take $\Theta(n)$ time for reallocation
- However, any sequence of $\Theta(n)$ operations takes $\Theta(n)$ time
- So each operation takes $\Theta(1)$ time “on average”

Amortized Analysis

- Data structure analysis technique to distribute cost over many operations
- Operation has **amortized cost** $T(n)$ if k operations cost at most $\leq kT(n)$
- “ $T(n)$ amortized” roughly means $T(n)$ “on average” over many operations
- Inserting into a dynamic array takes $\Theta(1)$ **amortized** time
- More amortization analysis techniques in 6.046!

Dynamic Array Deletion

- Delete from back? $\Theta(1)$ time without effort, yay!
- However, can be very wasteful in space. Want size of data structure to stay $\Theta(n)$
- **Attempt:** if very empty, resize to $r = 1$. Alternating insertion and deletion could be bad...
- **Idea!** When $r < r_d$, resize array to ratio r_i where $r_d < r_i$ (e.g., $r_d = 1/4$, $r_i = 1/2$)
- Then $\Theta(n)$ cheap operations must be made before next expensive resize
- Can limit extra space usage to $(1 + \varepsilon)n$ for any $\varepsilon > 0$ (set $r_d = \frac{1}{1+\varepsilon}$, $r_i = \frac{r_d+1}{2}$)
- Dynamic arrays only support dynamic **last** operations in $\Theta(1)$ time
- Python List `append` and `pop` are amortized $O(1)$ time, other operations can be $O(n)$!
- (Inserting/deleting efficiently from front is also possible; you will do this in PS1)

Data Structure	Operation, Worst Case $O(\cdot)$				
	Container	Static	Dynamic		
	build(X)	get_at(i) set_at(i, x)	insert_first(x) delete_first()	insert_last(x) delete_last()	insert_at(i, x) delete_at(i)
Array	n	1	n	n	n
Linked List	n	n	1	n	n
Dynamic Array	n	1	n	$1_{(a)}$	n

MIT OpenCourseWare
<https://ocw.mit.edu>

6.006 Introduction to Algorithms
Spring 2020

For information about citing these materials or our Terms of Use, visit: <https://ocw.mit.edu/terms>